

Tugas Analisis Multimedia: Image (Citra Digital)

- **Mata Kuliah:** Pengolahan Citra
 - **Nama:** Mahsa Yafi Nabilah
 - **NIM:** 122400094
-

Deskripsi Tugas

Tugas ini bertujuan untuk memahami **representasi dasar data citra digital (image)** melalui praktik langsung memuat data, visualisasi komponen warna, serta melakukan analisis spasial sederhana menggunakan berbagai teknik dasar pengolahan citra.

Anda akan bekerja dengan satu atau beberapa gambar (foto diri, objek, atau lingkungan sekitar) untuk:

- Mengamati struktur data piksel dan channel warna (RGB, Grayscale, HSV, dsb.)
- Menganalisis perbedaan hasil visualisasi antar representasi warna
- Melakukan eksplorasi sederhana terhadap transformasi citra (cropping, filtering, edge detection, dll.)
- Menyimpulkan pengaruh setiap tahap pemrosesan terhadap persepsi visual

Fokus tugas ini adalah pada **pemahaman konsep representasi spasial citra digital** dan **interpretasi hasil visualisasi**, bukan pada manipulasi kompleks atau penerapan model pembelajaran mesin.

Soal 1 — Cropping dan Konversi Warna

- Ambil sebuah gambar diri Anda (*selfie*) menggunakan kamera atau smartphone.
- Lakukan **cropping secara manual** untuk menghasilkan dua potongan:
 - Cropping **kotak persegi pada area wajah**.
 - Cropping **persegi panjang pada area latar belakang**.
- Resize hasil crop menjadi **920×920 piksel**.
- Konversi gambar menjadi **grayscale** dan **HSV**, lalu tampilkan ketiganya berdampingan.
- Tambahkan **anotasi teks** berisi nama Anda di atas kepala pada gambar hasil crop.
 - Gaya teks (font, warna, posisi, ukuran, ketebalan) **dibebaskan**.
- Jelaskan efek **cropping** dan **perubahan warna** menggunakan **Markdown**.

```
# =====  
# SOAL 1
```

```

# Cropping dan Konversi Warna
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# MEMBUAT FOLDER HASIL
# =====

os.makedirs("results_ws4/soal1", exist_ok=True)

# =====
# PATH GAMBAR
# =====

img_path = os.path.join(os.getcwd(), 'assets_ws4', 'selfie.jpeg')

# =====
# MEMBACA GAMBAR
# =====

img = cv2.imread(img_path)

# =====
# CEK GAMBAR
# =====

if img is None:
    print("Gambar gagal dibaca")
else:
    print("Gambar berhasil dibaca")

    # =====
    # KONVERSI BGR -> RGB
    # =====

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # =====
    # UKURAN GAMBAR
    # =====

    print("Ukuran gambar :", img_rgb.shape)

    # =====
    # MENAMPILKAN GAMBAR ASLI

```

```

# =====

plt.figure(figsize=(8,8))
plt.imshow(img_rgb)
plt.title("Gambar Asli")
plt.axis("off")
plt.show()

# =====
# CROPPING MANUAL
# =====
# FORMAT:
# gambar[y1:y2, x1:x2]
# SESUAIKAN JIKA HASIL BELUM PAS

# Crop wajah
face_crop = img_rgb[100:600, 150:650]

# Crop background
bg_crop = img_rgb[150:650, 700:1200]

# =====
# CEK HASIL CROP
# =====

print("Ukuran crop wajah :", face_crop.shape)
print("Ukuran crop background :", bg_crop.shape)

# =====
# RESIZE
# =====

face_crop = cv2.resize(face_crop, (920, 920))
bg_crop = cv2.resize(bg_crop, (920, 920))

# =====
# TAMBAH TEKS
# =====

face_bgr = cv2.cvtColor(face_crop, cv2.COLOR_RGB2BGR)

cv2.putText(
    face_bgr,
    "Mahsa Yafi Nabilah",
    (100, 80),
    cv2.FONT_HERSHEY_SIMPLEX,
    1.2,
    (255, 0, 0),
    3
)

```

```

face_crop = cv2.cvtColor(face_bgr, cv2.COLOR_BGR2RGB)

# =====
# KONVERSI WARNA
# =====

gray = cv2.cvtColor(face_crop, cv2.COLOR_RGB2GRAY)

hsv = cv2.cvtColor(face_crop, cv2.COLOR_RGB2HSV)

# =====
# MENYIMPAN HASIL
# =====

cv2.imwrite(
    "results_ws4/soal1/face_crop.png",
    cv2.cvtColor(face_crop, cv2.COLOR_RGB2BGR)
)

cv2.imwrite(
    "results_ws4/soal1/background_crop.png",
    cv2.cvtColor(bg_crop, cv2.COLOR_RGB2BGR)
)

cv2.imwrite(
    "results_ws4/soal1/grayscale.png",
    gray
)

cv2.imwrite(
    "results_ws4/soal1/hsv.png",
    cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
)

# =====
# VISUALISASI RGB - GRAYSCALE - HSV
# =====

plt.figure(figsize=(18,6))

# RGB
plt.subplot(1,3,1)
plt.imshow(face_crop)
plt.title("RGB")
plt.axis("off")

# Grayscale
plt.subplot(1,3,2)
plt.imshow(gray, cmap='gray')

```

```

plt.title("Grayscale")
plt.axis("off")

# HSV
plt.subplot(1,3,3)
plt.imshow(hsv)
plt.title("HSV")
plt.axis("off")

plt.tight_layout()
plt.show()

# =====
# TAMPILKAN BACKGROUND CROP
# =====

plt.figure(figsize=(6,6))
plt.imshow(bg_crop)
plt.title("Background Crop")
plt.axis("off")
plt.show()

print("=====")
print("Soal 1 berhasil dijalankan")
print("Hasil tersimpan di results_ws4")
print("=====")

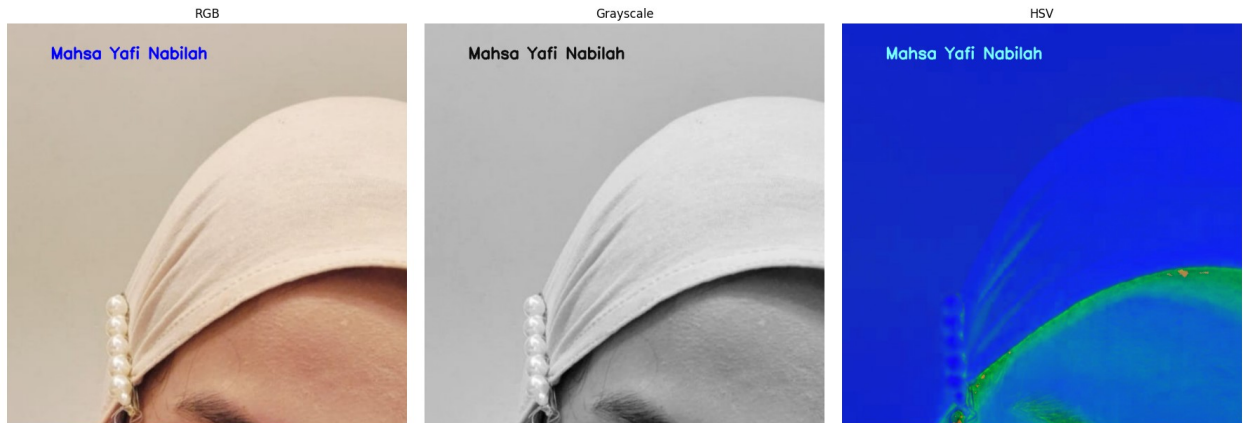
```

Gambar berhasil dibaca
 Ukuran gambar : (1600, 900, 3)

Gambar Asli



Ukuran crop wajah : (500, 500, 3)
Ukuran crop background : (500, 200, 3)



Background Crop



```
=====
Soal 1 berhasil dijalankan
Hasil tersimpan di results_ws4
=====
```

Analisis Cropping dan Konversi Warna

Cropping dilakukan untuk memfokuskan area tertentu pada citra, yaitu wajah dan latar belakang. Dengan cropping, informasi yang tidak diperlukan dapat dikurangi sehingga analisis visual menjadi lebih fokus.

Konversi grayscale menghilangkan informasi warna dan hanya mempertahankan intensitas cahaya. Hal ini membuat detail tekstur dan pencahayaan lebih terlihat.

Konversi HSV memisahkan informasi warna (Hue), tingkat kejenuhan (Saturation), dan kecerahan (Value), sehingga lebih mudah digunakan untuk segmentasi warna dan analisis pencahayaan.

Soal 2 — Manipulasi Channel Warna RGB

- Gunakan gambar hasil crop dari Soal 1.
- Konversikan gambar ke ruang warna **RGB**.
- Lakukan manipulasi channel warna dengan cara:
 - **Naikkan intensitas channel merah sebanyak 50 poin** (maksimum 255).
 - **Turunkan intensitas channel biru sebanyak 30 poin** (minimum 0).
- Teknik atau cara menaikkan/menurunkan intensitas **dibebaskan**, asalkan logis dan hasilnya terlihat.
- Gabungkan kembali channel warna dan **simpan gambar hasil modifikasi dalam format .png**.
- **Tampilkan histogram per channel (R, G, B)** untuk gambar asli dan hasil modifikasi menggunakan `matplotlib.pyplot.hist`.
- Jelaskan dampak perubahan RGB pada warna gambar dalam sel **Markdown**.

```
# =====
# SOAL 2
# Manipulasi Channel RGB
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

os.makedirs("results_ws4/soal2", exist_ok=True)

# Membaca gambar hasil crop
img = cv2.imread("results_ws4/soal1/face_crop.png")

img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```



```

# =====
# MEMISAHKAN CHANNEL
# =====

r, g, b = cv2.split(img_rgb)

# =====
# MANIPULASI CHANNEL
# =====

# Menambah merah
r_new = np.clip(r.astype(np.int16) + 50, 0, 255)

# Mengurangi biru
b_new = np.clip(b.astype(np.int16) - 30, 0, 255)

# Menggabungkan kembali
modified = cv2.merge([
    r_new.astype(np.uint8),
    g,
    b_new.astype(np.uint8)
])

# =====
# MENYIMPAN HASIL
# =====

cv2.imwrite(
    "results_ws4/soal2/rgb_modified.png",
    cv2.cvtColor(modified, cv2.COLOR_RGB2BGR)
)

# =====
# VISUALISASI GAMBAR
# =====

plt.figure(figsize=(12,6))

plt.subplot(1,2,1)
plt.imshow(img_rgb)
plt.title("Gambar Asli")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(modified)
plt.title("Gambar Modifikasi RGB")
plt.axis("off")

plt.tight_layout()
plt.show()

```

```

# =====
# HISTOGRAM
# =====

colors = ['red', 'green', 'blue']

plt.figure(figsize=(14,6))

# Histogram asli
plt.subplot(1,2,1)

for i, color in enumerate(colors):
    plt.hist(
        img_rgb[:, :, i].ravel(),
        bins=256,
        color=color,
        alpha=0.5
    )

plt.title("Histogram RGB Asli")
plt.xlabel("Intensitas")
plt.ylabel("Jumlah Piksel")

# Histogram modifikasi
plt.subplot(1,2,2)

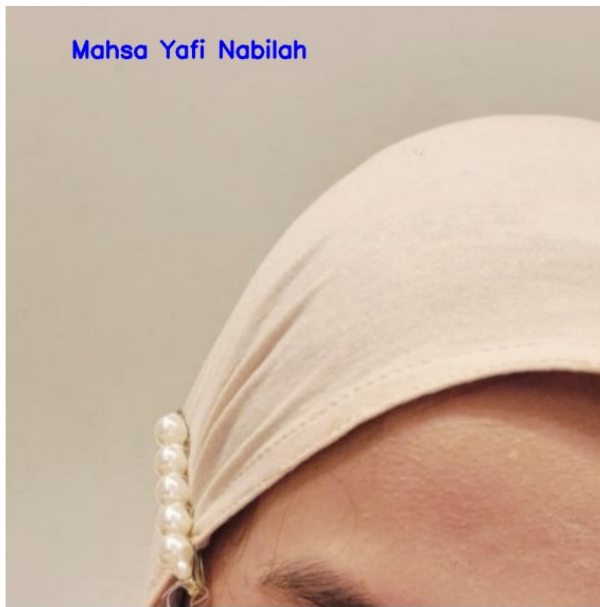
for i, color in enumerate(colors):
    plt.hist(
        modified[:, :, i].ravel(),
        bins=256,
        color=color,
        alpha=0.5
    )

plt.title("Histogram RGB Modifikasi")
plt.xlabel("Intensitas")
plt.ylabel("Jumlah Piksel")

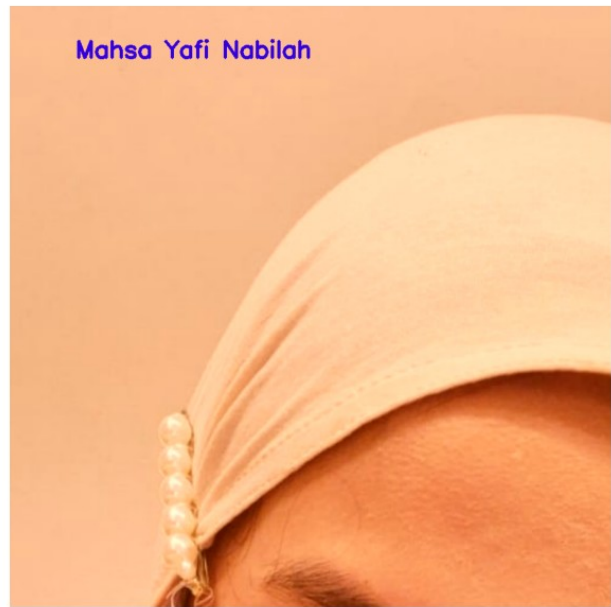
plt.tight_layout()
plt.show()

```

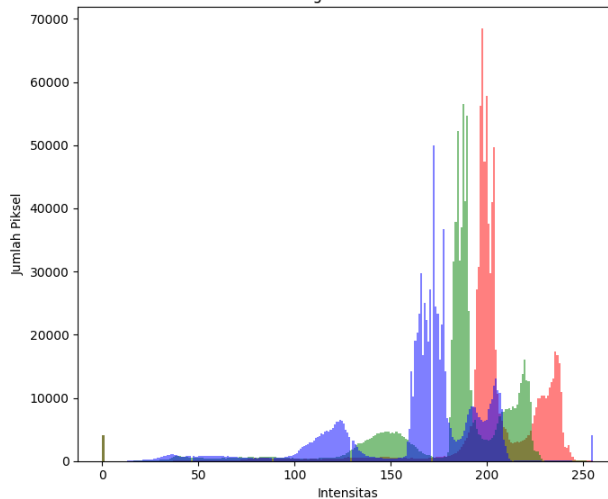
Gambar Asli



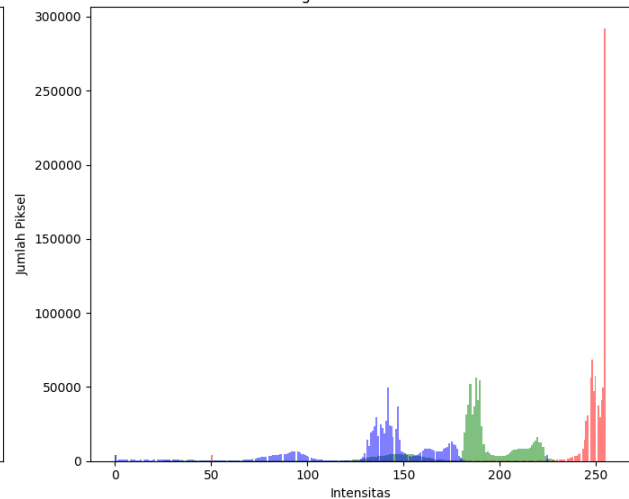
Gambar Modifikasi RGB



Histogram RGB Asli



Histogram RGB Modifikasi



Analisis Manipulasi RGB

Peningkatan channel merah membuat gambar terlihat lebih hangat dan dominan kemerahan. Sebaliknya, pengurangan channel biru membuat nuansa dingin pada gambar berkurang.

Histogram menunjukkan distribusi intensitas warna berubah setelah manipulasi. Distribusi merah bergeser ke kanan karena intensitas meningkat, sedangkan distribusi biru bergeser ke kiri akibat penurunan intensitas.

Soal 3 — Deteksi Tepi dan Filter Citra

- Ambil gambar **objek dengan background bertekstur** (misalnya kain bermotif, jerami, atau batu).
- Terapkan **edge detection (Canny)** dan tampilkan hasilnya.

- Lakukan **thresholding** dengan nilai ambang tertentu (bebas Anda tentukan) agar hanya objek utama yang tersisa.
- Buat **bounding box** di sekitar objek hasil segmentasi (boleh manual atau otomatis).
- Terapkan **filter blur** dan **filter sharpening**, lalu **bandingkan hasil keduanya**.
- Jelaskan bagaimana setiap filter memengaruhi detail gambar dalam format **Markdown**.

```
# =====
# SOAL 3
# Edge Detection dan Filter
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# MEMBUAT FOLDER HASIL
# =====

os.makedirs("results_ws4/soal3", exist_ok=True)

# =====
# PATH GAMBAR
# =====

img_path = os.path.join(os.getcwd(), 'assets_ws4', 'kalku.jpeg')

# =====
# MEMBACA GAMBAR
# =====

img = cv2.imread(img_path)

# =====
# CEK GAMBAR
# =====

if img is None:
    print("Gambar gagal dibaca")
else:
    print("Gambar berhasil dibaca")

    # =====
    # BGR -> RGB
    # =====

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```

# =====
# MENAMPILKAN GAMBAR ASLI
# =====

plt.figure(figsize=(6,6))
plt.imshow(img_rgb)
plt.title("Gambar Asli")
plt.axis("off")
plt.show()

# =====
# EDGE DETECTION CANNY
# =====

edges = cv2.Canny(img, 100, 200)

# =====
# THRESHOLDING
# =====

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, thresh = cv2.threshold(
    gray,
    120,
    255,
    cv2.THRESH_BINARY
)

# =====
# CONTOUR DAN BOUNDING BOX
# =====

contours, _ = cv2.findContours(
    thresh,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

# Mengecek contour tersedia
if len(contours) > 0:

    largest_contour = max(contours, key=cv2.contourArea)

    x, y, w, h = cv2.boundingRect(largest_contour)

    boxed = img_rgb.copy()

    cv2.rectangle(
        boxed,

```

```

        (x, y),
        (x+w, y+h),
        (0, 255, 0),
        4
    )

else:
    boxed = img_rgb.copy()

# =====
# FILTER BLUR
# =====

blur = cv2.GaussianBlur(img_rgb, (9,9), 0)

# =====
# FILTER SHARPEN
# =====

kernel = np.array([
    [0,-1,0],
    [-1,5,-1],
    [0,-1,0]
])

sharp = cv2.filter2D(img_rgb, -1, kernel)

# =====
# MENYIMPAN HASIL
# =====

cv2.imwrite(
    "results_ws4/soal3/canny.png",
    edges
)

cv2.imwrite(
    "results_ws4/soal3/threshold.png",
    thresh
)

cv2.imwrite(
    "results_ws4/soal3/bounding_box.png",
    cv2.cvtColor(boxed, cv2.COLOR_RGB2BGR)
)

cv2.imwrite(
    "results_ws4/soal3/blur.png",
    cv2.cvtColor(blur, cv2.COLOR_RGB2BGR)
)

```

```

cv2.imwrite(
    "results_ws4/soal3/sharpen.png",
    cv2.cvtColor(sharp, cv2.COLOR_RGB2BGR)
)

# =====
# VISUALISASI
# =====

fig, ax = plt.subplots(2,3, figsize=(18,10))

ax[0,0].imshow(img_rgb)
ax[0,0].set_title("Gambar Asli")
ax[0,0].axis("off")

ax[0,1].imshow(edges, cmap='gray')
ax[0,1].set_title("Canny Edge")
ax[0,1].axis("off")

ax[0,2].imshow(thresh, cmap='gray')
ax[0,2].set_title("Threshold")
ax[0,2].axis("off")

ax[1,0].imshow(boxed)
ax[1,0].set_title("Bounding Box")
ax[1,0].axis("off")

ax[1,1].imshow(blur)
ax[1,1].set_title("Blur")
ax[1,1].axis("off")

ax[1,2].imshow(sharp)
ax[1,2].set_title("Sharpen")
ax[1,2].axis("off")

plt.tight_layout()
plt.show()

print("=====")
print("Soal 3 berhasil dijalankan")
print("=====")

```

Gambar berhasil dibaca

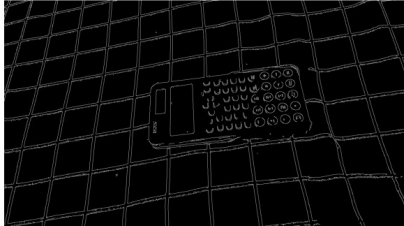
Gambar Asli



Gambar Asli



Canny Edge



Threshold



Bounding Box



Blur



Sharpen



=====
Soal 3 berhasil dijalankan
=====

Analisis Filter Citra

Filter blur mengurangi detail dan noise dengan cara menghaluskan perubahan intensitas piksel. Hasil gambar menjadi lebih lembut namun detail kecil berkurang.

Filter sharpening meningkatkan kontras lokal sehingga tepi objek terlihat lebih tajam. Namun sharpening berlebihan dapat memperkuat noise pada citra.

Soal 4 — Deteksi Wajah dan Filter Digital Kreatif

- Ambil gambar diri Anda dengan ekspresi wajah **netral**.

- Lakukan **deteksi wajah dan landmark** menggunakan salah satu dari:
 - **MediaPipe**, atau
 - **Dlib**, atau
 - **OpenCV**.
- Buat **overlay filter digital kreatif** karya Anda sendiri, misalnya:
 - topi, kumis, masker, helm, aksesoris, atau bentuk unik lainnya.
 - Filter boleh dibuat dari **gambar eksternal (PNG)** *atau* digambar langsung (misal bentuk lingkaran, garis, poligon, dll).
- Pastikan posisi overlay menyesuaikan **landmark wajah** dengan logis.
- **Gunakan alpha blending sebagai saran** agar hasil tampak lebih natural.
- Tampilkan perbandingan antara **gambar asli** dan **hasil dengan filter**.
- Jelaskan bagaimana Anda menghitung posisi overlay dan tantangan yang dihadapi selama implementasi (gunakan **Markdown**).

```
import cv2
import mediapipe as mp
import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# FOLDER OUTPUT
# =====
os.makedirs("results_ws4/soal4", exist_ok=True)

# =====
# PATH FILE
# =====
face_path = os.path.join("assets_ws4", "formal.jpeg")
glasses_path = os.path.join("assets_ws4", "sunglasses.png")

# =====
# BACA GAMBAR
# =====
img = cv2.imread(face_path)
glasses = cv2.imread(glasses_path, cv2.IMREAD_UNCHANGED)

# =====
# CEK GAMBAR
# =====
if img is None:
    raise FileNotFoundError("Foto wajah tidak ditemukan!")

if glasses is None:
```

```

        raise FileNotFoundError("PNG kacamata tidak ditemukan!")

print("Semua gambar berhasil dibaca")

# =====
# PREPROCESS
# =====
h, w = img.shape[:2]
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# =====
# MEDIAPIPE FACE MESH
# =====
mp_face_mesh = mp.solutions.face_mesh

with mp_face_mesh.FaceMesh(
    static_image_mode=True,
    max_num_faces=1,
    min_detection_confidence=0.5
) as face_mesh:

    results = face_mesh.process(img_rgb)

    output = img.copy()

    # =====
    # DETEKSI LANDMARK
    # =====
    if results.multi_face_landmarks:

        for face_landmarks in results.multi_face_landmarks:

            left_eye = face_landmarks.landmark[33]
            right_eye = face_landmarks.landmark[263]

            x1, y1 = int(left_eye.x * w), int(left_eye.y * h)
            x2, y2 = int(right_eye.x * w), int(right_eye.y * h)

            # =====
            # UKURAN KACAMATA
            # =====
            glasses_width = abs(x2 - x1) + 120
            scale = glasses_width / glasses.shape[1]
            glasses_height = int(glasses.shape[0] * scale)

            resized = cv2.resize(glasses, (glasses_width,
            glasses_height))

            # =====
            # POSISI

```

```

# =====
x = x1 - glasses_width // 7
y = y1 - glasses_height // 2

# clamp biar tidak keluar frame
x = max(0, x)
y = max(0, y)

# =====
# OVERLAY ALPHA BLENDING (VECTORIZED)
# =====
for i in range(glasses_height):
    for j in range(glasses_width):

        if y + i >= h or x + j >= w:
            continue

        alpha = resized[i, j, 3] / 255.0

        for c in range(3):
            output[y + i, x + j, c] = (
                alpha * resized[i, j, c] +
                (1 - alpha) * output[y + i, x + j, c]
            )

    else:
        print("Wajah tidak terdeteksi")

# =====
# SAVE OUTPUT
# =====
output_path = "results_ws4/soal4/filter_glasses.png"
cv2.imwrite(output_path, output)

# =====
# VISUALISASI
# =====
output_rgb = cv2.cvtColor(output, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(14, 7))

plt.subplot(1, 2, 1)
plt.imshow(img_rgb)
plt.title("Gambar Asli")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(output_rgb)
plt.title("Filter PNG (MediaPipe)")
plt.axis("off")

```

```
plt.tight_layout()
plt.show()

print("=====")
print("Filter PNG berhasil diterapkan")
print("Hasil disimpan di results_ws4/soal4/")
print("=====")
```

Semua gambar berhasil dibaca

Gambar Asli



Filter PNG (MediaPipe)



```
=====
Filter PNG berhasil diterapkan
Hasil disimpan di results_ws4/soal4/
=====
```

Analisis Overlay Digital

Posisi overlay kacamata ditentukan menggunakan koordinat landmark mata kiri dan mata kanan yang diperoleh dari MediaPipe Face Mesh. Jarak antara kedua mata digunakan untuk menghitung ukuran overlay agar tetap proporsional terhadap wajah pengguna.

Posisi kacamata ditempatkan sedikit di atas area mata sehingga terlihat lebih natural dan sesuai dengan bentuk wajah. Pada implementasi ini, filter digital menggunakan file PNG transparan sehingga diperlukan proses alpha blending agar overlay dapat menyatu dengan gambar asli tanpa menghilangkan detail wajah.

Tantangan utama implementasi adalah menjaga posisi dan ukuran overlay tetap sesuai ketika ukuran wajah berubah atau ketika posisi wajah tidak benar-benar simetris pada gambar.

Soal 5 — Perspektif dan Peningkatan Kualitas Citra

- Ambil gambar **objek datar** seperti karya tangan di kertas, tulisan di papan tulis, atau foto produk di meja dengan kondisi pencahayaan atau sudut yang tidak ideal.
- Lakukan **preprocessing** untuk memperbaiki tampilan agar lebih rapi dan jelas, dengan langkah-langkah:
 - Konversi ke **grayscale**.
 - **Koreksi perspektif (transformasi homografi)** menggunakan **4 titik manual** agar objek terlihat sejajar dan tidak terdistorsi.
 - Terapkan **thresholding adaptif atau Otsu** (pilih salah satu, dan jelaskan alasan pilihan Anda).
- Tampilkan **setiap tahap pemrosesan dalam satu grid** agar mudah dibandingkan.
- Jelaskan fungsi masing-masing tahap dan bagaimana teknik ini meningkatkan kualitas visual citra (gunakan **Markdown**).

```
# =====
# SOAL 5
# Perspektif dan Peningkatan Kualitas
# =====

import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

# =====
# MEMBUAT FOLDER HASIL
# =====

os.makedirs("results_ws4/soal5", exist_ok=True)

# =====
# PATH GAMBAR
# =====

img_path = os.path.join(os.getcwd(), 'assets_ws4', 'objek.jpeg')

# =====
# MEMBACA GAMBAR
# =====

img = cv2.imread(img_path)
```

```

# =====
# CEK GAMBAR
# =====

if img is None:
    print("Gambar gagal dibaca")
else:
    print("Gambar berhasil dibaca")

    # =====
    # KONVERSI BGR -> RGB
    # =====

    img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    # =====
    # MENAMPILKAN GAMBAR ASLI
    # =====

    plt.figure(figsize=(6,6))
    plt.imshow(img_rgb)
    plt.title("Gambar Asli")
    plt.axis("off")
    plt.show()

    # =====
    # GRAYSCALE
    # =====

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # =====
    # TITIK PERSPEKTIF MANUAL
    # =====
    # SESUAIKAN TITIK DENGAN POSISI OBJEK

    pts1 = np.float32([
        [100,120], # kiri atas
        [500,100], # kanan atas
        [120,650], # kiri bawah
        [520,670]  # kanan bawah
    ])

    pts2 = np.float32([
        [0,0],
        [600,0],
        [0,800],
        [600,800]
    ])

```

```

])

# =====
# VISUALISASI TITIK PERSPEKTIF
# =====

point_img = img_rgb.copy()

for point in pts1:
    x, y = point.astype(int)

    cv2.circle(
        point_img,
        (x,y),
        10,
        (255,0,0),
        -1
    )

# =====
# TRANSFORMASI PERSPEKTIF
# =====

matrix = cv2.getPerspectiveTransform(pts1, pts2)

warp = cv2.warpPerspective(
    img_rgb,
    matrix,
    (600,800)
)

# =====
# THRESHOLD OTSU
# =====

warp_gray = cv2.cvtColor(warp, cv2.COLOR_RGB2GRAY)

_, otsu = cv2.threshold(
    warp_gray,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# =====
# MENYIMPAN HASIL
# =====

cv2.imwrite(
    "results_ws4/soal5/grayscale.png",

```

```

    gray
)

cv2.imwrite(
    "results_ws4/soal5/perspective_points.png",
    cv2.cvtColor(point_img, cv2.COLOR_RGB2BGR)
)

cv2.imwrite(
    "results_ws4/soal5/warp.png",
    cv2.cvtColor(warp, cv2.COLOR_RGB2BGR)
)

cv2.imwrite(
    "results_ws4/soal5/otsu.png",
    otsu
)

# =====
# VISUALISASI GRID
# =====

fig, ax = plt.subplots(2,2, figsize=(14,10))

# Gambar asli
ax[0,0].imshow(point_img)
ax[0,0].set_title("Titik Perspektif")
ax[0,0].axis("off")

# Grayscale
ax[0,1].imshow(gray, cmap='gray')
ax[0,1].set_title("Grayscale")
ax[0,1].axis("off")

# Perspective transform
ax[1,0].imshow(warp)
ax[1,0].set_title("Perspective Transform")
ax[1,0].axis("off")

# Threshold otsu
ax[1,1].imshow(otsu, cmap='gray')
ax[1,1].set_title("Threshold Otsu")
ax[1,1].axis("off")

plt.tight_layout()
plt.show()

print("=====")
print("Soal 5 berhasil dijalankan")

```



```
print("Hasil tersimpan di results_ws4")  
print("=====")
```

Gambar berhasil dibaca

Gambar Asli



Titik Perspektif



Grayscale



Perspective Transform



Threshold Otsu



```
=====
Soal 5 berhasil dijalankan
Hasil tersimpan di results_ws4
=====
```

Analisis Perspektif dan Thresholding

Transformasi perspektif digunakan untuk memperbaiki distorsi akibat sudut pengambilan gambar yang miring. Dengan homografi, objek terlihat lebih sejajar dan proporsional.

Metode Otsu dipilih karena mampu menentukan nilai threshold optimal secara otomatis berdasarkan distribusi intensitas piksel. Hasilnya membuat objek utama lebih jelas dibanding latar belakang.

Bantuan AI (ChatGPT) Screenshot 2026-05-24 193510.png

References

- Github https://github.com/acl21/Selfie_Filters_OpenCV.git
- ChatGPT (OpenAI) - digunakan untuk bantuan penjelasan dan pelacakan kesalahan (debugging)

Aturan Umum Pengerjaan

- Kerjakan secara **mandiri**.
- Bantuan AI (seperti ChatGPT, Copilot, dsb.) diperbolehkan **dengan bukti percakapan** (screenshot / link / script percakapan).
- Source code antar mahasiswa harus berbeda.
- Jika mendapat bantuan teman, tuliskan nama dan NIM teman yang membantu.
- Plagiarisme akan dikenakan sanksi sesuai aturan akademik ITERA.
- Cantumkan seluruh **credit dan referensi** yang digunakan di bagian akhir notebook.
- Penjelasan setiap soal ditulis dalam **Markdown**, bukan di dalam komentar kode.

Aturan Pengumpulan

- Semua file kerja Anda (notebook .ipynb, gambar, dan hasil) **wajib diunggah ke GitHub repository tugas sebelumnya**.
 - Gunakan struktur folder berikut di dalam repo Anda:

```
/Nama_NIM_Repo/ # Nama repo sebelumnya
├── assets_ws4/   # berisi semua gambar atau video asli
(input)
├── results_ws4/  # berisi semua hasil modifikasi dan
output
├── worksheet4.ipynb
└── NIM_Worksheet4.pdf
```

- File yang dikumpulkan ke **Tally** hanya berupa **hasil PDF** dari notebook Anda, dengan format nama:

NIM_Worksheet4.pdf

- Pastikan notebook telah dijalankan penuh sebelum diekspor ke PDF.
 - Sertakan tautan ke repository GitHub Anda di bagian atas notebook atau di halaman pertama PDF.
-

□ Catatan Akhir

Worksheet 4 ini bertujuan mengasah pemahaman Anda tentang manipulasi citra digital secara praktis. Gunakan kreativitas Anda untuk menghasilkan hasil visual yang menarik dan penjelasan konseptual yang jelas.